CA2 项目速读：异常感知的路由与封禁控制器

> 读完这份文档（约 15 分钟），你应该能说清：题目要做什么、网络长什么样、控制器每 2 秒做了哪几件事、实验怎么做、关键数字是多少、文件在哪。细节查 `report/report.pdf`（最终报告，16 页）和 `PLAN.md`。

1. 一句话

我们写了一个 Ryu SDN 控制器：发现 ICMP 洪泛攻击 → 在所有交换机上封掉攻击源 → 把被攻击链路的代价调高 → 让共用这条链路的合法流量临时绕行，攻击结束后再回来。

题目（CEG5101 CA2，满分 30）分 5 个任务：

任务	分值	要做什么	我们的结果在哪
T1 拓扑	5	在 MiniEdit 里搭出给定拓扑并运行	报告第 2 节，图 1、图 2；`miniedit/my_topology.mn`
T2 检测与封禁	5	统计每个源的速率，用 EMA 基线检测异常，封掉攻击源	报告第 3.2 节、第 5.1 节
T3 动态代价	10	链路代价随负载和“是否被攻击”变化	报告第 3.3 节、第 6.2 节
T4 改路由	10	合法流量绕开被攻击链路（case a/b/c）	报告第 3.4 节、第 5 节
T5 性能评估	—	拓扑截图 + 代价、丢包、吞吐随时间的图 + 解释	报告第 6 节（fig1–fig5），扩展实验在第 7 节

评分看三点：概念理解、设计推理、结果解读。

2. 网络

- 6 台交换机 s1–s6，每台挂一台主机（hi 接 si，IP 10.0.0.i/24）。主机链路 40 Mbps、15 ms。
- 11 条交换机之间的链路，都是 40 Mbps，时延（ms）就是静态代价 C0：

链路	C0	链路	C0
s1-s2	5	s2-s5	12
s2-s4	6	s2-s6	14
s3-s6	7	s4-s5	15
s1-s4	8	s1-s5	18
s3-s5	9	s5-s6	20
s2-s3	10		

- 我们保护的合法流：h1→h6，最短路 s1-s2-s6（代价 19），备用 s1-s2-s3-s6（22，只贵 3）。
- 三种攻击（hping3 ICMP 洪泛，约 4,800 包/秒，打满 40 Mbps）：

case	攻击	实际走的链路	和 h1→h6 共用吗
a	h2→h3	s2-s3	不共用 → 不该改路由
b	h2→h6	s2-s6	共用 → 应该改路由（全场的主角）
c	h5→h6	s5-s3-s6（16 比直连 s5-s6 的 20 便宜）	不共用 → 不该改路由

3. 控制器每 2 秒做什么

代码：`controller/integrated_controller.py`（Ryu 应用）+ `controller/sdn_logic.py`（参数、代价公式、Dijkstra、检测器，不依赖 Ryu，有 18 个单元测试）。它合并了老师给的两份参考代码：一份会选路但看不见攻击，一份会封禁但不会挪流量。

一个循环 = 测量 → 检测 → 定价 → 选路，每 2 秒一次（实际间隔 2.4 秒：等 2 秒，再等 0.4 秒收统计回复）。

1. 测量
    - FlowStats：每个源在它的入口交换机上的包速率（pkt/s）。不统计 ICMP echo reply，否则被攻击的 h6 会因为回包太多而被当成攻击者。
    - PortStats：每条链路的速率 R（EMA 平滑）。
2. 检测（T2）
    - 每个源有一个 EMA 基线 b（权重 0.3）和波动 σ。
    - 阈值 T = max(3b, b + 4σ, 500 pkt/s)：3b 是题目给的规则，b+4σ 跟着这个源自己的波动走，500 是下限。
    - 先比较，再更新基线，异常样本不学进基线。参考代码是先更新再比较，实际阈值会变成约 21 倍。

    - 每个源最开始的 5 个样本只学习、不报警（warm-up）。有链路时延时，实验脚本自动改成 12 个样本（见第 6 节）。
3. 封禁
    - 超过阈值的源：在全部 6 台交换机装优先级 5000 的 drop 规则（按源 IP 匹配）。
    - 攻击停止、安静 10 秒后自动解封。
4. 定价（T3）
    - C = C0 + α·R/Rmax + β·A，α = 10，β = 30，Rmax = 40 Mbps。
    - A = 1 表示这条链路被攻击源走过。只要攻击者还在撞 drop 规则（速率 ≥ 0.5T），A 就保持 1，停了之后再保持 10 秒。
    - 负载项在封禁后几个周期就消失了，β·A 是“记忆”，它让被攻击链路在攻击期间一直很贵：s2-s6 的平台值 = (14+30)/14 = 3.14×。
5. 选路（T4）
    - 每个流比较“留在当前路径”和“最好的备选路径”。
    - what-if：评估备选路径时，把这个流自己的负载加到它还没走的链路上，避免来回振荡。
    - 备选要便宜超过 2（margin）才换。
    - 先装出口、再装入口（s6 → s3 → s2 → s1），换路时不丢包。
6. 其他
    - ARP 由控制器代答，广播只发给主机端口，所以网状拓扑不需要生成树，也不会有广播风暴。
    - LLDP 发现全部 11 条链路后才开始转发；首次发现完整后就锁定“已就绪”，否则断一条链路会导致全网 ARP 失效（D13，断链实验发现的 bug）。

设计决策 D1–D14 的完整列表（问题 → 做法 → 理由）在 `PLAN.md` 第 1 节。

4. 实验怎么做

`experiments/run_experiment.py` 一条命令跑完：

1. 起 Ryu 和 Mininet，`pingall` 两次让所有主机被学到。
2. h1→h6 发 10 Mbps UDP（iperf，约 890 包/秒），h1 每 0.5 秒 ping 一次 h6（测 RTT）；h2–h6 各有一路 1 pps 的背景 ping（给检测器建立基线）。
3. 时间线：基线 40 秒 → case a 洪泛 25 秒 → 恢复 45 秒 → case b → 恢复 → case c → 恢复，共约 255 秒。
4. 独立测量：h6 的 iperf 每秒报告（丢包、吞吐）、ping（RTT）、h6 上的 tcpdump（攻击包有没有漏进来）、流表 dump、控制器自己的 CSV 日志。
5. `analysis/plot_results.py` 画 fig1–fig5，生成 `summary.md`。

做了哪些运行：

运行	环境	目录
R1、R2、R3	云端容器（OVS 用户态、没有链路时延）。报告的主要数字来自这里	`results/validation-abc`、`run1-abc`、`run2-abc`
β 实验	β = 0 和 β = 2，只跑 case b	`results/exp-beta0-b`、`exp-beta2-b`
case c 覆盖	把 s5-s6 的 C0 改成 8，让攻击真的走 s5-s6	`results/exp-c-s5s6-cost8`
断链	s2-s6 断 20 秒	`results/exp-linkfail-s2s6`
VM 重跑	官方 Mininet 2.3.0 VM，内核态 OVS + 真实链路时延	`results/vm-delay-abc`

5. 关键数字（背熟这些）

指标	数值
检测延迟（9 次洪泛）	1.5–3.8 秒（1–2 个采样周期）
封禁后	封禁 1 秒之后没有任何攻击包到达受害者
case b 检测前最坏一秒丢包	48.4%（R1），三次运行 48.4 / 56.6 / 34.4%
R1 case b 检测前逐秒丢包	24 / 47 / 48 / 27%
检测前以外	h1→h6 始终 10.5 Mbps、0% 丢包
改路由	只有 case b 改，就在封禁那一刻；留下代价 58.8 vs 换走 30.0
回切	洪泛结束后约 14 秒（13.6–14.9 秒），比较值 24.5 vs 30.1；每次运行正好 2 次改路由
代价平台	s2-s3 4.0×、s2-s6 3.14×、s3-s5 4.33×、s3-s6 5.29×，即 (C0+30)/C0
β 实验	β = 0 或 2 时不改路由，但封禁后丢包同样是 0 → 止损靠封禁，β 防的是封禁漏掉的流量（如伪造源）
断链	7 毫秒内改路由，71,515 个包只丢 11 个
VM（有时延）	RTT 基线 99.8 ms（推算 98），绕行 109 ms；检测 2.2 / 1.6 / 3.8 秒，行为和容器一致

全场主线：所有丢包都发生在检测前那 1.5–3.8 秒里；封禁结束丢包，β 让流量离开被攻击的链路，滞后机制让它只回切一次。

6. 局限（被问到要主动说）

- 检测延迟是最弱的一环：轮询间隔 2.4 秒，第一个样本可能只覆盖部分洪泛。
- 按源 IP 封禁，对伪造源或低速分布式攻击无效；那时改路由是剩下的防线。
- 封禁期间攻击主机的正常流量也被丢掉（10 秒后自动解封）。
- 受害者自己的接入链路（如 s6-h6）绕不开，只能靠封禁（case c）。
- 阈值下限 500、hold 时间、β = 30 是针对这个拓扑调出来的，不是推导出来的。
- warm-up：一个合法流如果在 warm-up 结束后才以高速率出现，会被误封。在 VM 上真的发生过（带时延时 pingall 变慢，h1 被误封），所以有时延时实验脚本自动用 12 个样本。
- R1–R3 在容器里跑（没有链路时延）；VM 有真实时延，但它是软件模拟的，洪泛弱一些，丢包数字不能直接比。

7. 文件地图

位置	是什么
`report/report.pdf`、`report.tex`	最终报告（英文，16 页）
`controller/`	控制器代码
`topology/ceg_mesh_topo.py`	题目给的拓扑
`miniedit/my_topology.mn`	MiniEdit 里搭的拓扑（与题目一致）；`my_topology_export.py` 是导出的脚本
`experiments/run_experiment.py`	一键实验
`analysis/plot_results.py`	画图和汇总
`tests/test_sdn_logic.py`	18 个单元测试
`results/`	每次运行的原始数据、图和 `summary.md`
`scripts/setup_ubuntu.sh`	一键安装（Mininet VM / Ubuntu）
`presentation/slides/`	幻灯片（单文件 HTML，图片和视频都嵌在里面，直接用 Chrome/Edge 打开）和讲义 PDF
`presentation/script/speaker_script.pdf`	演讲稿（英文，按页）
`presentation/video/`	三段视频：MiniEdit 搭建、case b 真实运行、R1 回放
`presentation/data/numbers.md`	幻灯片上每个数字的出处
`demo/`	现场演示脚本（`live_demo.sh` 和四个监视窗格）
`docs/`	速读、Q&A 题库、现场演示方案、技术支持（Windows / Mac）
`PLAN.md`	中文执行计划和全部设计决策

8. 自己跑一遍（Mininet VM 上）

```
        bash scripts/setup_ubuntu.sh
        export RYU_MANAGER=~/ryu-venv/bin/ryu-manager PLOT_PYTHON=~/ryu-
venv/bin/python
        ~/ryu-venv/bin/python -m unittest discover -s tests      # 18 个测试
        sudo -E python3 experiments/run_experiment.py            # 约 5 分钟，结果
在 results/<时间戳>/
```

现场演示（四格画面，一条命令跑完 a、b、c 并出结果表）：`bash demo/live_demo.sh`，只跑 case b 用 `bash demo/live_demo.sh quick`。讲解和分工见 `docs/Live_Demo_Runbook_zh.pdf`。

只想看控制器：`~/ryu-venv/bin/ryu-manager --observe-links controller/integrated_controller.py`，然后另开一个终端用 MiniEdit 打开 `miniedit/my_topology.mn` 点 Run，在 CLI 里 `pingall`。

注意：Mininet 2.3.0 自带的 MiniEdit 在 Python 3 下 File → Save 会存出空文件，要先把 `~/mininet/examples/miniedit.py` 里两处 `open(fileName, 'wb')` 改成 `'w'`。

9. 最可能被问的 5 个问题

1. 为什么 β = 30？小一点会怎样？ 直觉上 β 要大于绕行多出的 3 再加 margin 2；算上本流自身负载，稳态下要大于约 7.7。β = 0 或 2 时不改路由。30 留了很大余量，是调出来的。
2. 封禁已经让丢包归零了，为什么还要改路由？ 封禁对伪造源无效；改路由保护的是封禁漏掉的流量。代价只是路径代价多 3（单程约多 3 ms），而且只在攻击期间。
3. 题目说 case c 打 s5-s6，为什么你们标记的是 s3-s5、s3-s6，h1 也没改路由？ 按 C0，h5→h6 走 s5-s3-s6（16 < 20），我们标记的就是攻击真正走过的链路；它们不在 h1→h6 上，所以不改路由是对的。把 s5-s6 改成 8 的实验里攻击确实走了 s5-s6，h1 仍然不动。
4. 为什么重写了 C0 表？ 参考代码的代价表对应一个 4 交换机拓扑，和我们的 6 交换机网格冲突（还缺 7 条链路）。我们保留“C0 = 链路时延”的原则，为 11 条链路重写了表。
5. 检测延迟从哪来，怎么缩短？ 2.4 秒一次轮询，要 1–2 个样本；缩短可以用更快的轮询或 sFlow 采样，代价是控制开销和误报。

完整的 26 题题库（英文答案 + 证据位置）在演示设计文档第 9 节。